TECHNICAL WHITEPAPER · EXECUTION ISOLATION

Ephemora Cell

The execution layer for untrusted, AI-generated code: capability-based WASM sandboxing with explicit CPU, memory, time, I/O, and filesystem limits - 8/8 attack vectors blocked, 0.46 ms warm invocation, one runtime dependency.

Version 1.0.0 · Apache-2.0 · Open Source ·

September 2026

GITHUB.COM/MICHAELS1011/EPHEMORA-CELL

The Problem

AI agents write code - and run it. That is exactly where the gap opens: how do you execute untrusted code without giving that code access to your host, your credentials, your network, or unlimited compute?

Containers alone do not answer that question. In a verified test run, a stock **python:3.12-slim** container blocked **0 of 8** typical attack vectors, while Ephemora Cell blocked **8 of 8** (live-verified; the verification scripts ship with the repository).

8/8
attack vectors blocked by Ephemora Cell (live-verified)

0/8
attack vectors blocked by a stock python:3.12-slim container

What Ephemora Cell Is

Ephemora Cell is an execution primitive - not an agent framework. It sits underneath agent stacks, MCP servers, and plugin systems, and runs each workload in a WebAssembly sandbox under enforced limits:

AI Agent → Tool / MCP → Ephemora Cell → WASM (wasmtime) → bounded result

Enforced limit	Default
CPU (fuel metering)	1,000,000 fuel (~13 fuel per iteration, calibration $R^2 = 1.000$)
Memory	128 MB hard cap
Wall clock	30 s (epoch interruption)
Network	Disabled - no socket APIs exposed via WASI
Filesystem	Deny-by-default; 14 dangerous directories blocked
stdout / stderr	10 KB cap
Threads / exec / fork	Unavailable inside the sandbox

On top of the sandbox: OS-level hardening in a worker process (rlimits, disk quota, I/O CPU watchdog, hard kill), I/O budgets against I/O denial-of-service, per-session named state, and an egress sidecar reference mediator.

Measured Performance

All figures below were measured on Apple M5 hardware with wasmtime 47 (n = 1000 unless stated otherwise) and are reproducible from the repository benchmark suite.

0.46 ms	427x	0.25 ms
warm pooled invocation, wall-clock median (p95: 0.60 ms)	cold-start advantage over Docker (170.63 ms vs 0.400 ms, n = 7, live-measured 2026-08-30)	Cell overhead vs raw WASI baseline (0.027 ms)

Fairness note: the 427x figure compares container cold start against an invoked WASM module - it is not a general-purpose performance claim, and we state it as such.

Security, Independently Checked

Isolation claims are only as good as their evidence. Ephemora Cell ships its attack tests and publishes its limits:

- **8/8 attack vectors blocked:** shell/fork/sockets, fsync, /etc/passwd read, symlink escape, threading, and environment leakage - verified live, versus 0/8 in a stock container.
- **External evaluation (arXiv 2509.11242)** tested 11 exploitation strategies against the sandbox: 8 fully blocked, 2 bounded by design (disk-DoS, high-frequency I/O), 1 permitted within budget (CPU-DoS) - openly documented in docs/security_posture.md.
- **Honesty as a principle:** Cell is an execution boundary, not a claim that guest code is trustworthy. The threat model and known limitations are public in the repository.

Engineering Quality

Claims about engineering quality are cheap; here the numbers are CI-enforced on every push:

379	85%	1
tests in the suite	statement coverage (84.55% measured), CI gate at 80%	runtime dependency: wasmtime - no framework zoo

- CI on every push: tests across Python 3.10/3.11/3.12, pip-audit, SBOM, bandit, and a fuzzing workflow.
- MCP server included: a dependency-free stdio server where tools are WASM modules; execution reports carry the wasmtime version as an auditable witness, and a signing primitive (ExecutionReport.sign(), SEP-2787-style) is present.
- Platforms verified: macOS (Apple Silicon), Ubuntu 24.04, and NVIDIA DGX Spark.

Release

Ephemora Cell v1.0.0 is published on PyPI (released 2026-08-30) under the Apache-2.0 license, installable with **pip install ephemora-cell**.

- Guest languages: Rust, Go, C, AssemblyScript, and Zig - compiled and executed in CI.
- Integration tests exist for LangGraph, CrewAI, AutoGen, the OpenAI Agents SDK, Semantic Kernel, and other agent stacks.
- Ephemora Cell is the open-source isolation layer, standalone with no Ephemora dependency; the Ephemora enterprise edition builds on Cell's isolation.

Getting Started

Cell installs as a single PyPI package with one runtime dependency and works from the terminal or inside an MCP client:

- **Install:** pip install ephemora-cell
- **Run a module with limits:** ephemora-cell run tool.wasm --json - the JSON report includes the security baseline; profiles such as --profile analytical are available.
- **MCP server:** ephemora-cell-mcp starts a dependency-free stdio server; register WASM tools from a directory with --tools-dir.
- **Tooling:** ephemora-cell inspect, benchmark, and build cover module inspection, measurement, and WASM builds for Rust, Go, C, AssemblyScript, and Zig.

GitHub: github.com/MichaelS1011/ephemora-cell · **PyPI:** pypi.org/project/ephemora-cell

Sources and Measurement Notes

All figures come from the repository itself: README.md, docs/performance.md, docs/security_posture.md, and BENCHMARK_RESULTS.md.

Older benchmark runs (agentic benchmark of 2026-08-06, "up to 148x") are flagged in the repository as predating the benchmark rework and possibly not reproducible; they are deliberately not used as marketing figures.

Coverage as measured: 84.55% against a CI gate of 80%. Whitepaper date: September 2026. All numbers reflect the code as released in v1.0.0.